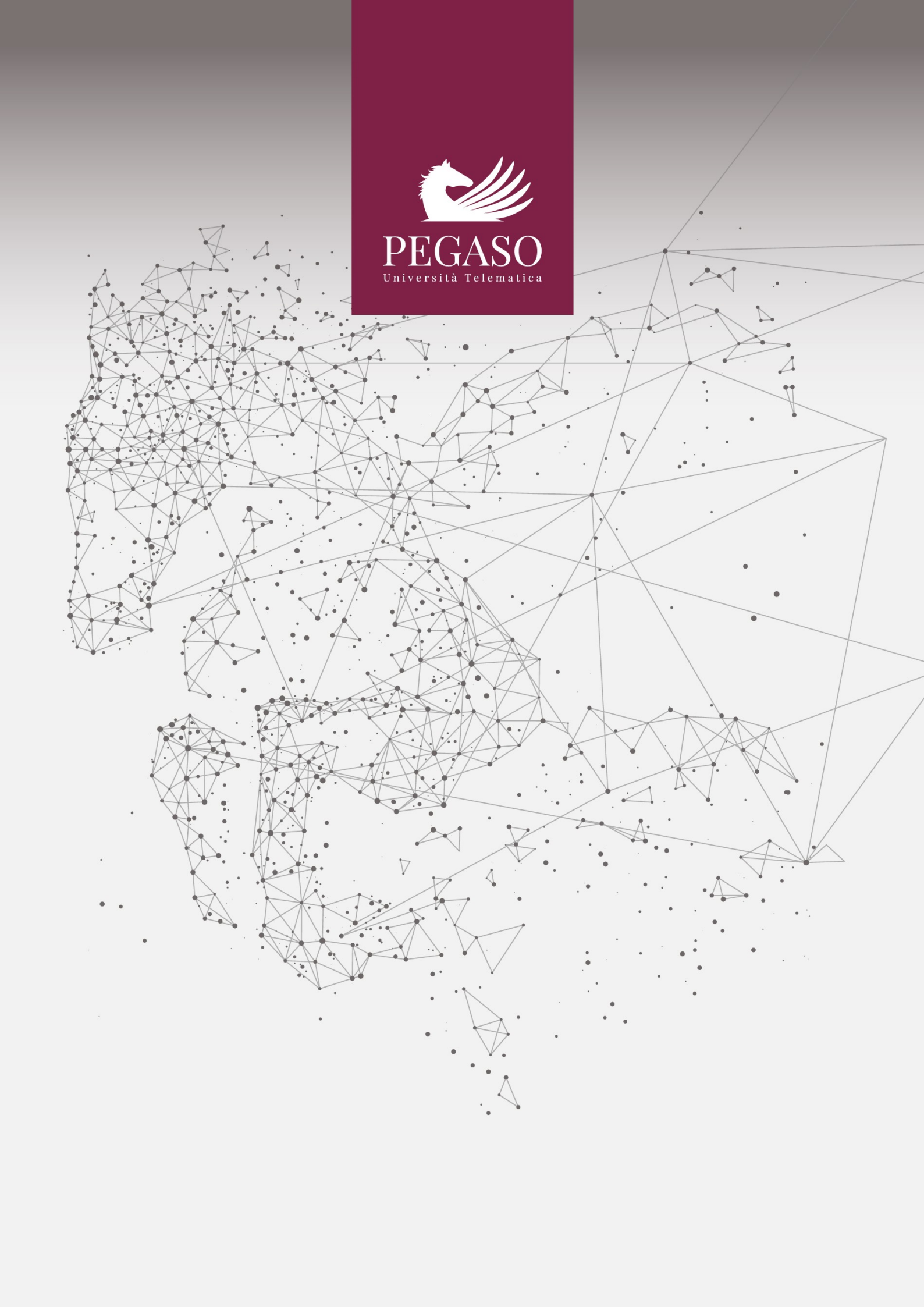

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE....3

2. INTRODUZIONE AI BIG DATA....4

3. HADOOP....6

3.1 COMPONENTI PRINCIPALI DI HADOOP....7
3.2 L'ECOSISTEMA DI HADOOP....9
3.3 CLUSTER E NODI DI HADOOP....11
3.4 MAPREDUCE....15

4. APACHE SPARK....18

BIBLIOGRAFIA....21

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si introducono gli aspetti fondamentali relativi ai Big Data, tra cui la regola delle 3V e i paradigmi di organizzazione di sistemi dei dati. Viene, inoltre, presentato e approfondito Apache Hadoop, uno dei primi framework realmente orientato ai Big Data. Vengono, infatti, illustrati i componenti principali di Hadoop, l'ecosistema Hadoop, i nodi di HDFS (il file system distribuito di Hadoop), i processi principali di YARN (il gestore di risorse di Hadoop) e l'algoritmo MapReduce per l'elaborazione di dati in cluster di nodi. Infine, viene introdotto Apache Spark, uno dei framework per Big Data completo che deriva da Hadoop. Ne vengono analizzati i vantaggi, i moduli principali e il funzionamento di un cluster Spark.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Introduzione ai Big Data

Tutte le certezze dell'analisi dati tradizionale sono andati in crisi di fronte ai **Big Data.** I Big Data sono uno dei fenomeni più significativi degli ultimi decenni e se ne inizia a parlare nei primissimi anni del 2000 quando venne formulata la cosiddetta **regola delle 3V**:

1. **Volume;**
2. **Velocità;**
3. **Varietà.**

Approfondiamo ora ciascuna delle 3V.

Volume

La quantità di dati da trattare è estremamente alta. Questo, però, non è l'unico fattore che caratterizza tale realtà, ma sicuramente è il più indicativo. Non a caso alla parola *data* è stato anteposto l'aggettivo *big.* Il rapporto di **tali volumi** con quelli con cui siamo abituati a trattare quotidianamente sui nostri sistemi di elaborazione non ha paragone.

Se i file con cui si ha a che fare a casa o in ufficio misurano Megabyte (10^6 byte) o al massimo Gigabyte (10^9 byte), gli **archivi Big Data** si misurano nell'ordine dei **Terabyte** (10^{12} byte) o **Petabyte** (10^{15} byte).

Velocità

Altro importante fattore è la rapidità con cui i **dati vengono accumulati**. Sono sempre più, infatti, i contesti in cui un flusso di dati senza sosta (*streaming*) affluisce ai server. Ciò ha portato alla necessità di **gestirne l'accumulazione eliminando perdite di dati, evitando il blocco dei sistemi e riducendo al minimo i tempi di latenza**. Casi di questo tipo possono riguardare, ad esempio, transazioni finanziarie o dispositivi IoT (Internet Of Things) con la costante trasmissione di eventi rilevati.

Varietà

A differenza dei classici archivi fortemente strutturati, i **dati** che vengono accumulati nei sistemi Big Data sono **altamente variegati**. Basta pensare ai post che si pubblicano continuamente sui social network: essi sono composti da testo, immagini, video e altri contenuti multimediali, in numero, quantità e ordine mutevole a caso.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Anche se questa legge rappresenta il nocciolo essenziale dei Big Data, di «V», nel tempo, ne sono state individuate molte altre. Una di queste è la **Veridicità dei dati.**

La quantità e la velocità di accumulazione devono, comunque, permettere **l'affidabilità delle informazioni**. Dati errati possono, infatti, non avere alcun valore e, in casi peggiori, portare a **valutazioni non corrette.** Tale aspetto non è poi facile da verificare quando i **dati vengono prodotti da essere umani.**

Grandi volumi a grande velocità caratterizzati da grande varietà portano necessariamente a problematiche di organizzazione di accoglienza dei dati in arrivo. È necessario, quindi:

1. capire come organizzare i dati;
2. prevedere i tipi di dati possibili;
3. decidere se strutturarli in arrivo o rielaborarli in seguito.

I più comuni paradigmi di organizzazione di sistemi dei dati sono tre:

1. suddivisione in **comparti stagni**, detti in genere *silos.* Ogni comparto dell'azienda mantiene i propri archivi e non ci sono condivisioni con gli altri. Questo paradigma è molto diffuso e può verificarsi per diversi motivi, tra cui **mancanza di interesse, riservatezza e protezione delle informazioni**;
2. **Datawarehouse**, ovvero archivi strutturati di dati. Si cerca di integrare tutti i dati dall'azienda in un unico sistema nella speranza di **ottimizzarne la gestione e sfruttarne sinergie e correlazioni**. È un sistema molto in voga negli ultimi decenni che può portare ad attività di analisi particolarmente interessanti. I dati all'interno, tuttavia, devono essere **strutturati al momento della loro introduzione;**
3. **Data Lake**, il *lago di dati*. È un'organizzazione per cui ogni dato in arrivo, di qualsiasi genere, viene accantonato in un unico storage senza alcuna elaborazione in ingresso. Si inverte così tutto ciò che gli archivi tradizionali trattano. Se si pensa classici database relazionali prevale il principio dello **schema on-write** (*strutturazione in scrittura*). Esiste uno schema di base per il database e i dati possono essere inseriti solo se si adattano a questa struttura.

Nel Data Lake prevale il paradigma inverso, lo **schema on-read**: i dati vengono accumulati e salvati e solo nel momento in cui li si vorrà utilizzare, si provvederà a **interpretarli e adattarli ad una struttura.**

Tale approccio, adottato da molte aziende, facilita l'acquisizione e il salvataggio di dati nonostante l'alta velocità e la varietà.

Nonostante le aziende si rivolgano, generalmente, ad uno specifico di questi approcci, prende sempre più piede la strada dell'**integrazione.** Un'azienda può avere bisogno di un Data Lake per l'accumulazione di dati, ma anche di un Datawarehouse per la loro analisi. **Tali sistemi possono convivere contemporaneamente** dedicandosi alla raccolta di dati diversi o costituendo fasi attigue dello stesso ciclo di vita.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Hadoop

Apache Hadoop è considerato uno dei primi **framework realmente orientato ai Big Data.**

Nato nel 2006, a pochi anni dalla definizione delle regole delle **3V**, trae ispirazione da due progetti di ricerca finanziati da Google:

- *Google File System* del 2003;
- *Google MapReduce* del 2004.

Entrambi i progetti sono stati significativi perché hanno fornito una risposta alle esigenze primarie di chi si occupava di Big Data:

1. **dove archiviare** grandi quantità di file;
2. **come elaborare** tali dati una volta archiviati.

Il logo di Apache Hadoop è riportato di seguito.



Sul sito ufficiale del progetto https://hadoop.apache.org/ Hadoop viene presentato come un framework open-source per la programmazione:

- **affidabile;**
- **scalabile;**
- **distribuita.**

Ne risulta che Hadoop non deve solo essere in grado di **gestire una grande mole di informazioni**, ma anche di farlo in maniera **sicura**, **senza perdite** di dati, con **prestazioni apprezzabili** e una **struttura di nodi** costantemente espandibile (anche in caso di migliaia di nodi).

PEGASO
Università Telematica

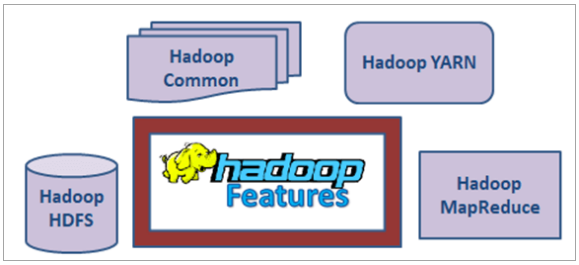
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3.1 Componenti principali di Hadoop

Hadoop è costituito da quattro componenti principali:

1. **Hadoop Common** – una serie di librerie condivise sfruttate da tutti i moduli di Hadoop in ogni possibile settore di applicazione;
2. **HDFS** (*Hadoop Distributed File System*) – un **file system distribuito, robusto e sicuro** che permette di distribuire tutti i dati immagazzinati in un **gran numero di nodi costituenti un cluster;**
3. **MapReduce** – il primo, ma non unico, algoritmo di elaborazione capace di elaborare dati in cluster;
4. **Hadoop YARN** (*Yet Another Resource Manager*) – strato specializzato nella gestione delle risorse. Costituisce un ambiente di elaborazione per *MapReduce*, ma anche di altri framework che possono essere innestati in Hadoop.

Nella Figura successiva, si riporta uno schema generale con i quattro componenti principali di Hadoop.



Più in dettaglio, **HDFS** è un **file system distribuito** che fa uso di supporti hardware. HDFS accomuna l'insieme di nodi che costituiscono il *cluster*. Pertanto, a differenza dei file system tradizionali, deve poter assicurare un **alto grado di affidabilità.**

Un *cluster* formato da numerose macchine può incorrere frequentemente in guasti, ma **tali eventi non possono condizionare le prestazioni di elaborazione né causare perdite di dati**.

L'affidabilità viene garantita dalla **suddivisione in blocchi dei singoli file** da 64 MB o da 128 MB che sono replicati su più nodi. Ciò permetterà di ottimizzare le operazioni di lettura e di **non avere mai una copia unica dei dati.** Una caratteristica di Hadoop è quella di essere **rack-aware**.

Ciò permette di **replicare blocchi di dati non solo su server diversi**, ma anche collocati in rack differenti.

Altro aspetto fondamentale di Hadoop è la **scalabilità che applica in un cluster**. La scalabilità è **orizzontale**, cioè si basa *sull'aumento del numero di nodi piuttosto che sul loro potenziamento.*

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Affinché tutte le operazioni di elaborazione e gestione vengano svolte con regolarità e successo è importante che nel **cluster ogni macchina abbia il proprio ruolo.**

Il sistema su cui installare Hadoop cambia in base allo scopo che si vuole perseguire. Infatti, se si vuole utilizzare Hadoop per la fase di «**produzione**», si ha la necessità di disporre di un **vero cluster in sede (o disponibile via rete)** al quale si potranno inviare i *job* preparati. Tale contesto può essere probabilmente già predisposto dall'azienda che ha richiesto la consulenza.

Se, invece, si vuole utilizzare Hadoop per studiarne il funzionamento (o semplicemente provarlo) è possibile procedere senza un vero e proprio *cluster.*

L'installazione di Hadoop non è particolarmente complessa, ma a volte si preferisce ricorrere a **macchine virtuali** già pronte che includono installazioni running di Hadoop e altri strumenti ad esso collegati. Un esempio di questa soluzione è fornito da Cloudera (www.cloudera.com).

Se si vuole procedere con una normale esecuzione di *job* su Hadoop esistono **tre modalità di installazione e funzionamento**:

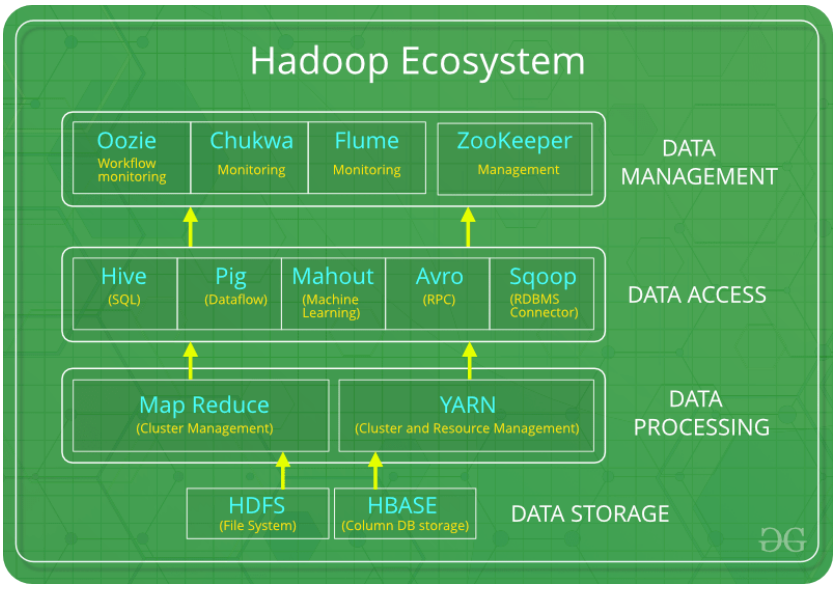
1. modalità **stand-alone** – tutto il funzionamento è racchiuso in un unico processo Java e viene utilizzato per il debugging del codice;
2. modalità **pseudo-distributed** – il funzionamento sarà suddiviso in più processi Java. Richiede un minimo di installazione, ma si ha il vantaggio di assistere all'attivazione sia di YARN, sia di HDFS e il tutto sarà racchiudibile in un'unica macchina (*single-node cluster*);
3. modalità **fully-distributed** – si dovranno usare più nodi (macchine reali o virtuali) in cui si crea un reale cluster Hadoop da estendere in base alle necessità.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3.2 L'ecosistema di Hadoop

Nel tempo Hadoop si è trasformato da sistema dotato soltanto di proprie competenze in merito alla conservazione di file e alla capacità di elaborazione, a centro di un **ecosistema** che lo ha arricchito di varie funzionalità (si veda lo schema riportato nella Figura seguente).



I «**progetti collaterali**» hanno compensato:

- mancanze del sistema;
- nuove esigenze dei programmatori;
- nuove sfide tecnologiche.

Vediamo, ora, solo alcuni dei principali progetti (anche se piuttosto numerosi) dell'**ecosistema Hadoop**. Nella presentazione essi saranno suddivisi per **categorie** in base al loro scopo principale.

La prima categoria dei progetti dell'ecosistema Hadoop è quella degli **strumenti per l'elaborazione dei dati.** Questi sono strumenti che si integrano su *MapReduce* o *YARN* e offrono vie alternative all'analisi dei dati a disposizione.

Tra di essi ci sono:

- **Spark** (spark.apache.org) – un framework dalle enormi potenzialità, ricco di moduli, vicino alle logiche più conosciute dal programmatore e aperto ad ogni possibile scenario (verrà approfondito più avanti);

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- **Tez** (tez.apache.org) – un modello di programmazione basato su *YARN* nato per offrire un'alternativa più flessibile e performante a *MapReduce*;
- **Drill** (drill.apache.org) – permette di effettuare interrogazioni in SQL *schema-free* su dati di diverso genere (ideale per Hadoop, NoSQL e ambienti Cloud);
- **Zeppelin** (zeppelin.apache.org) – strumento che propone *notebook* in stile web per eseguire lavorazioni su Big Data in vari linguaggi tra cui *SQL* e *Scala;*
- **Impala** (impala.apache.org) – altro ambiente ispirato al linguaggio SQL che parallelizza l'esecuzione sui nodi di un cluster Hadoop.

La seconda categoria di progetti dell'ecosistema Hadoop è quella degli **strumenti per la gestione di dati su HDFS**:

- **Hive** (hive.apache.org) – strumento ideato da Facebook che permette di elaborare dati in un formalismo stile SQL senza necessità di programmare;
- **Pig** (pig.apache.org) – strumento per operazioni di manipolazione di dati che sfrutta script espressi in un proprio linguaggio chiamato *Pig latin;*
- **Hbase** (hbase.apache.org) – si tratta a tutti gli effetti di un database NoSQL, di tipo chiave/valore che trova fondamento in *HDFS.*

La terza categoria di progetti dell'ecosistema Hadoop è quella degli **strumenti per il trasferimento di dati da e verso Hadoop** che permettono l'aggregazione di informazioni da altri ambienti sia in modalità *real-time* sia *batch*:

- **Sqoop** (sqoop.apache.org) – rappresenta il ponte tra il mondo dei database relazionali e Hadoop. Permette, infatti, una migrazione di dati tra Hadoop e database in entrambi i sensi;
- **Kafka** (kafka.apache.org) – sistema molto diffuso per la trasmissione di flussi di dati utilizzato in molti contesti oltre che in congiunzione con Hadoop;
- **Storm** (storm.apache.org) – funziona come una sorta di pipeline basata su *YARN* ed è impiegato spesso per dati *real-time*;
- **Flume** (flume.apache.org) – particolarmente usato per il flusso di dati di *log*, è rivolto a trasferimento di grandi quantità di dati non strutturati;
- **Spark Streaming** – una delle componenti di *Apache Spark* dedicato, in particolare, allo streaming dei dati.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La quarta categoria di progetti dell'ecosistema Hadoop è quella degli **strumenti per la gestione del cluster**:

- **Ambari** (ambari.apache.org) – offre capacità complete di gestione di un cluster Hadoop;
- **Zookeeper** (zookeeper.apache.org) – è uno strumento di coordinamento per gestire una molteplicità di nodi offrendo, tra l'altro, anche la possibilità di una condivisione di metadati;
- **Oozie** (oozie.apache.org) – si tratta essenzialmente di uno *scheduler* per flussi di lavoro in grado di coordinare la sequenza di operazioni che portano al completamento di un'operazione complessa.

La quinta categoria di progetti dell'ecosistema Hadoop è quella degli **strumenti dedicati al machine learning** (data la loro particolarità costituiscono una categoria a sé stante):

- **Mahout** (mahout.apache.org) – uno dei principali progetti di Machine Learning ed elaborazioni matematiche basato su Hadoop;
- **Spark** (spark.apache.org/mllib) – si ritrova ancora il nome di Spark in quanto nel cuore di questo framework esistono librerie specifiche per il Machine Learning che lo rendono anche in questo senso perfettamente integrabile con Hadoop.

3.3 Cluster e nodi di Hadoop

Anche se per sperimentare la programmazione (o lavorare su una installazione Hadoop) in *cluster* non è indispensabile conoscerne il funzionamento interno, è molto utile sapere *come questo framework lavora* e quali sono i *principali processi che lo attivano.*

Hadoop lavora in **cluster**, cioè una **moltitudine di nodi**. I *nodi del cluster* possono essere per lo più di due tipologie:

1. **master;**
2. **slave.**

I nodi master **coordinano le attività** degli slave.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Essenzialmente, avviare Hadoop significa mettere in moto i vari processi che costituiscono i servizi *HDFS* e *YARN.*

- **HDFS** si articola intorno ai nodi *NameNode*, *DataNode* e *Secondary NameNode;*
- **YARN** si basa su *ResourceManager* e *NodeManager*.

Considerando la presenza sia di *nodi master* che di *nodi slave*, nel cluster potremmo distinguere tali processi in base ai ruoli:

- Tra i **nodi master** avremo:
  - *NameNode* e *Secondary NameNode* per **HDFS**;
  - *ResourceManager* per **YARN**.
- Tra i **nodi slave** avremo:
  - *DataNode* per **HDFS**;
  - *NodeManager* per **YARN**.

Questi i ruoli previsti per ciascun nodo di HDFS:

1. ***NameNode***

Il *NameNode* ha il ruolo fondamentale di gestire il funzionamento degli altri nodi (risiede, infatti, nel *server principale*). Esso è il recettore **di qualsiasi richiesta che viene inoltrata al sistema HDFS** e conserva la mappa della distribuzione dei vari blocchi di file sui nodi.

Tale memorizzazione viene operata su un **file principale che prende il nome di *fsimage*** e viene poi completata con la scrittura di una serie di *log* di modifiche su un ulteriore file. Altro compito del *NameNode* è *tenere traccia periodicamente dello stato di salute* degli altri nodi del cluster mediante il segnale **heartbeat.**

2. ***DataNode***

I *DataNode* sono i nodi deputati alla **conservazione dei blocchi di file** e le cui tracce, come detto, vengono conservati dal *NameNode*. Essi si occupano di **eseguire le operazioni di lettura e scrittura** di file richieste dai client, ma anche quelle di **replica dei blocchi** di dati tra nodi diversi.

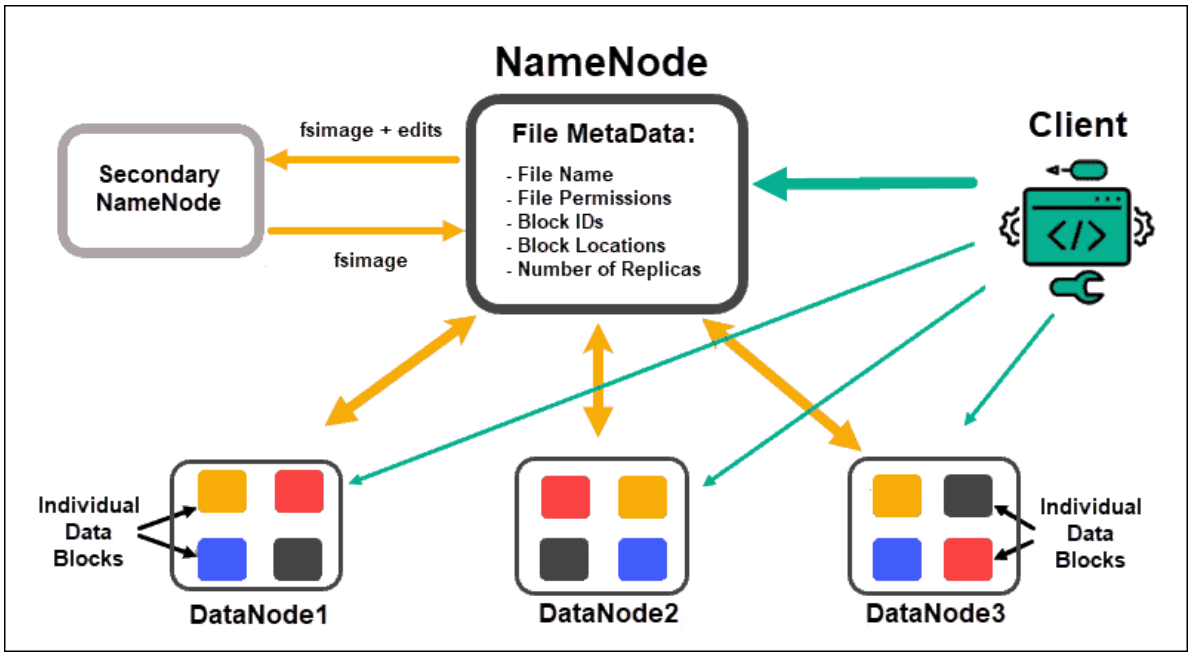
PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. ***Secondary NameNode***

Il *Secondary NameNode* si occupa di supportare il *NameNode* nello svolgimento delle operazioni. Periodicamente preleva dal *NameNode* il file **fsimage** e i *log* di modifica fondendoli in un unico file che restituisce al *NameNode*. Il *NameNode* potrà usarlo al suo prossimo riavvio per accelerare le sue operazioni.

Nella Figura seguente, si riporta uno schema logico dei nodi appena descritti e delle loro interazioni tra di essi e con l'esterno.



YARN, invece, si basa su due tipi di **processi principali**:

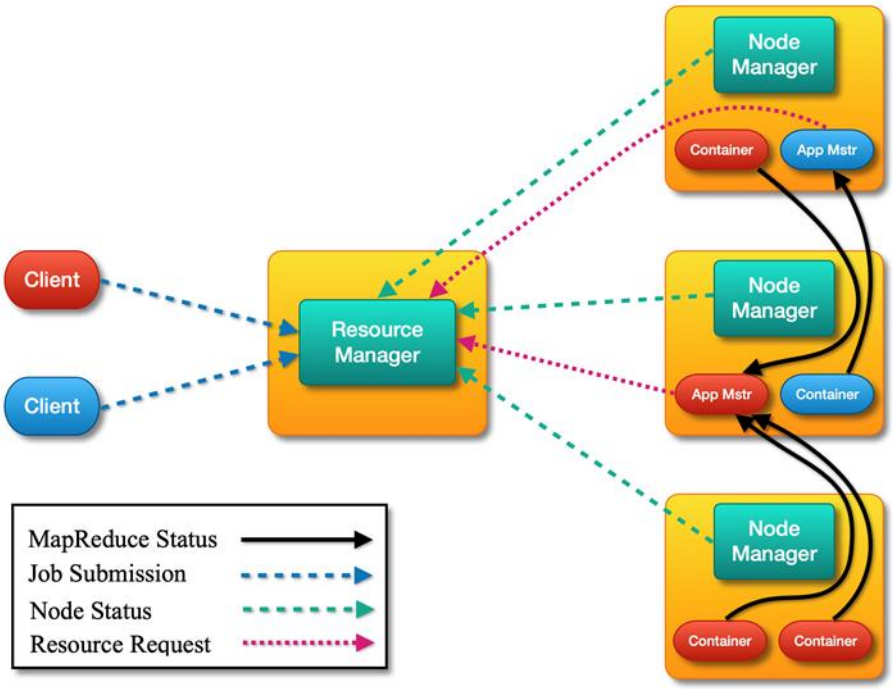
1. ***Resource manager***, un componente fondamentale di *coordinamento* di cui esiste una sola istanza nel cluster. Esso possiede una **mappatura completa degli altri nodi** e conosce la loro collocazione nonché le loro caratteristiche tecniche. Ne verifica, inoltre, lo stato di salute mediante **heartbeat** e mette tali informazioni a disposizione mediante API REST, interfaccia web o operazioni da riga di comando. Da tale processo di YARN inizia la schedulazione delle varie attività applicative che saranno poi svolte nei **nodi slave;**

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. ***Node manager***, in esecuzione in ogni nodo slave, riveste un ruolo fondamentale in varie attività. All'**avvio del cluster** si registra presso il *Resource manager* e da inizio ad una serie di attività informative sul nodo. Se uno dei servizi sotto la sua responsabilità **non funziona più**, il *Node Manager* lo comunica al *Resource manager* che può così rimanere informato. Il *Node Manager* riferisce, infine, sulle attività in corso sulle risorse disponibili nel nodo.

Nella Figura seguente, si riporta lo schema delle attività dei due processi principali di YARN nella gestione dei nodi e nei rapporti con i client.



L'**accesso al file system di HDFS** può essere condotto in due modi:

1. da riga di comando con il tool *hdfs* incluso nell'installazione di Hadoop;
2. mediante *WebAPI* da eseguire attraverso un client *HTTP*.

Ad ogni modo, le operazioni che possiamo svolgere in HDFS sono le stesse che si possono svolgere su un normale file system:

- elencare file e cartelle in una data posizione;
- calcolare lo spazio utilizzato nella cartella;

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- stampare in output il contenuto di un file;
- creare una nuova cartella;
- eseguire l'upload di un file da posizione locale ad HDFS;
- eseguire il download di un file da HDFS a macchina locale;
- copiare e spostare un file internamente a HDFS;
- cancellare un file o una cartella su HDFS.

Sapere come utilizzare i file in HDFS è molto importante proprio perché i file sono la **principale fonte dati** per le elaborazioni di Hadoop. Tipicamente i dati vengono accumulati su HDFS nel tempo e, quando necessario, vengono lanciati task di elaborazione che li coinvolgono. Il riferimento ai file verrà fatto in base agli indirizzi HDFS e sarà, poi, compito del *NameNode* rintracciarne i blocchi sui vari *DataNode* dove sono dislocati.

HDFS svolge, inoltre, un importante ruolo di archivio per elaborazioni operate da vari sistemi in grado di collegarsi ad esso. In generale, se si conosce **l'indirizzo del NameNode di HDFS**, si può accedere direttamente al percorso.

A titolo di esempio, si supponga di avere sul nostro elaboratore un'installazione *pseudo-distributed* di Hadoop il cui *NameNode* sia in ascolto all'indirizzo localhost, porta 9000. Sarà possibile accedere al file */archivio/testo.txt* con l'URL:

hdfs://localhost:9000/archivio/testo.txt

3.4 MapReduce

Un modulo fondamentale di Hadoop è il **MapReduce** che serve principalmente ad **elaborare grandi quantità di dati**. L'algoritmo di MapReduce ha dimostrato come in presenza di problematiche inerenti i Big Data (come la numerosità dei dati), le modalità di analisi dei dati devono necessariamente essere **riconsiderate**. Qualsiasi operazione che coinvolge grandi quantità di dati distribuiti su molti nodi deve, infatti, essere risolta scindendola in una **serie di fasi e aggregandone i risultati.** L'elaborazione risulta, pertanto, **parallelizzata** poiché viene eseguita contemporaneamente sui vari nodi che possiedono i dati.
Il lavoro di **MapReduce** si fonda sulla predisposizione di **due funzioni principali:**

1. una denominata **map;**
2. una denominata **reduce.**

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

MapReduce si articola sulle seguenti **quattro fasi**:

1. i **dati di partenza** (input dell'algoritmo) vengono resi disponibili nel cluster mediante HDFS;
2. la **fase di map** viene eseguita *parallelamente* su più nodi. Si leggono tutti i dati in input producendo come output una serie di *coppie chiave/valore*. Il valore viene calcolato durante il **map** e può trattarsi di numeri, stringhe o altri tipi di dato. Il valore calcolato servirà per ottenere il risultato finale dopo il **reduce**, mentre la **chiave**, invece, **serve per etichettare i valori**;
3. la fase di **reduce** riceve l'output del *map* con i valori suddivisi in vari gruppi in base alle chiavi associate ad essi. Si occupa, quindi, di svolgere operazioni di sintesi su ogni singolo gruppo (es. analisi statistiche, conteggi);
4. i risultati prodotti nella fase di *reduce* vengono salvati su HDFS e costituiscono l'output del processo.

Solitamente un solo task **MapReduce** non è sempre sufficiente per raggiungere l'obiettivo.

Molto spesso, infatti, il lavoro con Hadoop si traduce nella definizione di **più task MapReduce concatenati** che sfruttano come input ognuno l'output del precedente.

Procediamo con un semplice esempio che illustra come il funzionamento appena descritto possa essere messo in pratica.

Supponiamo:

- di avere **file contenenti una gran mole di numeri distribuiti sui nodi di un cluster HDFS**;
- che il nostro scopo sia quello di **sommare distintamente i numeri pari e dispari** (obiettivo semplice, ma significativo come esempio).

Da un punto di vista generale l'algoritmo che seguiremo è il seguente:

- nel **map** associamo ad ogni numero una chiave con valore PARI o DISPARI;
- nel **reduce** i numeri verranno automaticamente raggruppati per chiave.

Pertanto, essendo solo due le chiavi, troveremo due gruppi di numeri (uno con i dispari e uno con i pari).

La fase di **map** leggerà in input i dati riga per riga dei vari blocchi e produrrà in output una coppia composta dalla stringa PARI o DISPARI come chiave e il numero stesso analizzato come valore. Per esempio, se la funzione di *map* legge in input i seguenti valori:

10

6

7

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

8

La funzione di *map* produrrà in output:

PARI,10

PARI, 6

DISPARI, 7

PARI, 8

Quindi, a fronte dei valori 10, 6 e 8 (tutti pari) sono state emesse dal *map* tre coppie con chiave PARI, mentre a fronte del valore 7 una sola coppia con chiave DISPARI.

Il reduce riceverà un input tutte queste **coppie**, le quali, prima dell'operazione di somma, **verranno ordinate e combinate per chiave**.

Possiamo immaginare questa preparazione come una suddivisione delle coppie in gruppi distinti in base ai valori della chiave (il gruppo legato alla stringa PARI e quello relativo al DISPARI). Dopo la suddivisione, al *reduce* non resterà che procedere con il **calcolo della somma dei numeri appartenenti a ogni gruppo.**

Ovviamente il problema presentato è molto semplice. Sommare distintamente i numeri pari e dispari di una lista è un esercizio elementare, ma occorre ricordare che **in presenza di Big Data** anche l'operazione più semplice va pianificata in funzione della grande quantità di dati da gestire e della loro distribuzione su più server.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Apache Spark

Apache Spark è un **framework per Big Data completo** in quanto offre:

- librerie di elaborazione alternative a *MapReduce;*
- librerie per lo streaming dei dati;
- librerie per il Machine Learning.

Spark nasce intorno al 2010 quando **i Big Data erano già diventati un'emergenza** per il mondo della produzione della ricerca, mentre Hadoop esisteva da alcuni anni ed erano già stati messi in luce i suoi pregi e i suoi limiti. Segue una Figura in cui è rappresentato il logo di Apache Spark.



Apache Spark nasce con l'obiettivo di **uniformare il settore dell'elaborazione di grandi quantità di dati distribuiti in cluster** e di migliorare, in genere, le condizioni di lavoro del programmatore.

I principali vantaggi per cui l'ambiente Spark si distingue sono cinque e, precisamente:

1. **velocità** - Spark è contraddistinto da alte prestazioni che sono di gran lunga superiori a quelle di Hadoop. Oltre a vari fattori di gestione delle attività, il framework fa largo uso di *cache in memoria* a differenza dei primi sistemi per Big Data che prediligevano l'utilizzo del disco anche per la memorizzazione di dati temporanei;
2. **multi-linguaggio** - Spark non vincola all'uso di un solo linguaggio e dispone di funzionalità offerte in *Scala* (il suo linguaggio di creazione), *Java, SQL, R* e *Python*. Ciò permette l'accesso a Spark da parte di programmatori provenienti da qualsiasi ambito;
3. **facile da usare** - osservando gli esempi, si nota che la programmazione Spark ricorda i tradizionali ambienti di programmazione che tutti i programmatori conoscono. Ciò permette di ridurre le barriere d'ingresso a nuovi programmatori;
4. **varie configurazioni di installazione possibili** – Spark può essere seguito in vari modi:

- in modalità *stand-alone;*
- oppure collocato su uno stack Hadoop, Apache Mesos o Kubernetes.

  Esistono, inoltre, diverse opportunità di utilizzarlo direttamente su Cloud;

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. **comunità molto attiva** – Spark, grazie al suo successo e alla sua adattabilità, viene seguito da una vasta comunità di programmatori per cui è facile trovare tutorial, documentazione, esempi e librerie realizzate da terze parti.

Il primo approccio a Spark richiede una chiara cognizione di cosa compone esattamente questo progetto. La struttura di Apache Spark è **modulare** e consta di più parti. Ciò contribuisce all'obiettivo di rendere Spark un ambiente unificato, inclusivo e che contempla ogni tipo di analisi.

Il substrato di base sul quale poggiano **quattro moduli** ruota attorno ai **Resilient Distributed Dataset (RDD)** strutture dati fondamentali in grado di lavorare in *modalità distribuita su cluster.* I **quattro moduli principali** di Apache Spark sono:

1. **Spark SQL** - una delle principali innovazioni che Spark ha introdotto è una modalità per lavorare con dati strutturati in maniera simile a quella che ogni programmatore segue nell'interazione con i **database relazionali**. Esso vede l'ingresso delle strutture dati *DataSet* e *DataFrame* (quest'ultima di Python) e quello del linguaggio SQL. Le elaborazioni da svolgere potranno, infatti, essere condotte con *funzioni* oppure mediante *query SQL*. La scelta dipenderà in buona parte delle preferenze del programmatore;
2. **Spark Streaming** - dedicato alla trattazione di dati in *real-time* che affluiscono catalizzati da soluzioni come Kafka, Flume e Amazon Kinesis. Tali contesti sono particolarmente delicati in quanto, se non trattati nei modi e nei tempi giusti, la gestione dei dati può risultare del tutto fallimentare;
3. **Mlib** - la libreria per il Machine Learning che include algoritmi e funzionalità finalizzati a offrire questo tipo di analisi in maniera scalabile e performante;
4. **GraphX** - la libreria per grafi che unifica tutte le modalità di analisi ed esplorazione di queste strutture dati in un unico ambiente di funzioni.

Come detto, in Spark, è possibile programmare in vari linguaggi. In questo corso continueremo ad usare Python anche con Spark. Le API per Python di Spark prendono il nome di **PySpark** e sono disponibili su **PyPI** (*Python Package Index*), il principale repository di librerie Python.

Con PySpark è possibile utilizzare le funzioni sia in modalità interattiva da console, sia tramite script. Trattandosi di una tecnologia per Big Data, Spark è naturalmente predisposto per lavorare in cluster. Con **PySpark**, però, almeno in fase di sperimentazione non servirà avere un cluster a disposizione. Infatti, ad ogni avvio del codice verrà lanciato **un servizio locale che svolgerà il ruolo di cluster**.

Come ultimo aspetto, analizziamo, molto brevemente, il funzionamento di un **cluster Spark**.

PEGASO
Università Telematica

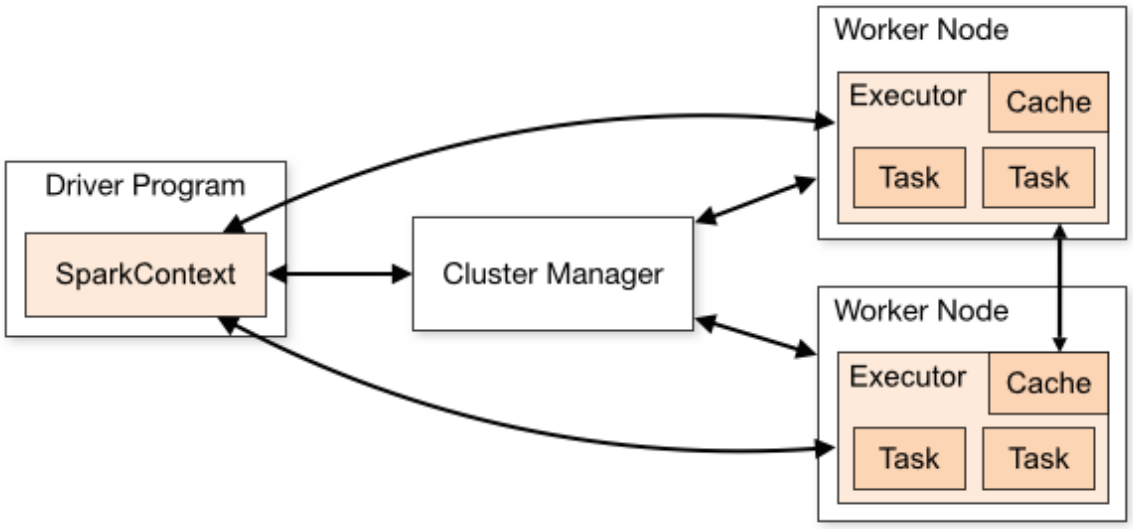
Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Sebbene non sia indispensabile per poter proseguire lo studio di Apache Spark, si introducono qui alcuni concetti che riguardano il **funzionamento di un cluster Spark** che torneranno utili come orientamento sia nella programmazione (ad esempio per comprendere il ruolo dello *SparkContext*), sia nella sperimentazione di installazioni multi-nodo.

In generale, un cluster Spark ruota attorno a **tre componenti principali**:

1. il **Cluster Manager** – elemento che svolge il coordinamento tra i vari nodi. Esso può essere offerto da *Spark* stesso, *YARN, Mesos* o *Kubernetes*. Tale componente offre quello strato di coesione tra i nodi appartenenti al cluster. È interessante notare come il **funzionamento di Spark sia del tutto indifferente al tipo di Cluster Manager scelto**;
2. il **Driver Program** dove risiede lo **SparkContext**. Lo *SparkContext* sarà l'elemento da cui verranno avviati i nostri programmi. Ogni operazione che verrà svolta con *PySpark* verrà avviata dall'interazione con lo *SparkContext* che rappresenterà il «ponte» con il cluster;
3. i **Worker Node** sono tutti i nodi del *cluster,* tra i quali sarà *parallelizzato* il lavoro. Ogni nodo attiva un **Executor** che riceve sia il codice da eseguire (come archivio JAR di Java o file Python), sia i *task* veicolati dal *Cluster Manager*. Ogni applicazione avvia i propri *Executor* risultando isolata dalle altre.

Nella Figura seguente, si riporta uno schema delle interazioni tra i componenti principali di Apache Spark.



PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Maggi G. (2020). Data science con Python. Edizioni LSWR.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).